LOW-LEVEL DESIGN (LLD)

Lecture 1 — Introduction to LLD

Detailed, structured study notes prepared from the lecture transcript

What these notes cover: the meaning of LLD, how LLD differs from DSA and HLD, the Quick Ride example, the three pillars of LLD, reusable algorithms, coupling, scalability, maintainability, reusability, and the lecture's concluding mental model.

Study approach: Learn the definitions first, then revisit the Quick Ride example. It demonstrates why solving an algorithmic problem is only one part of designing a complete application.

1. Lecture overview and learning goals

The lecture introduces Low-Level Design (LLD) as the step that turns algorithmic problem-solving knowledge into the structure of a practical software application. Knowing Data Structures and Algorithms (DSA) is useful, but a real application also needs well-defined objects, relationships, interactions, security considerations, and code that can be extended and maintained.

- Understand what LLD means and why it is needed after learning DSA.
- Distinguish an isolated algorithmic problem from the design of a complete application.
- Use the Quick Ride ride-booking platform as a running example.
- Learn the three pillars emphasised in the lecture: scalability, maintainability, and reusability.
- Understand the difference between LLD and High-Level Design (HLD).
- Remember how DSA, LLD, and HLD contribute to building software.

Prerequisite mentioned by the teacher: C++ is sufficient to follow the series; the teacher also says Java source code will be provided. The series is presented as beginner-friendly and intended to cover LLD concepts in depth.

2. What is Low-Level Design (LLD)?

Definition: LLD is concerned with the internal code-level structure of an application—what objects or entities it contains, how those objects relate to one another, and how they collaborate to implement the application's behaviour.

The teacher explains LLD by starting from DSA. DSA commonly addresses isolated problems, such as searching for an element or sorting an array. LLD takes the relevant solutions and places them within the design of a complete application.

2.1 DSA solves isolated problems

- **Searching:** If an element must be found in an array, an algorithm such as linear search or binary search may be used. Binary search is appropriate when the data is sorted and the search conditions are met; linear search can be used without sorted data.
- **Sorting:** Sorting an array is another isolated problem. Examples named in the lecture include Quick Sort, Insertion Sort, and Merge Sort.

These algorithms solve particular tasks. By themselves, they do not define the whole application around those tasks.

2.2 LLD puts solutions into an application

A complete application needs more than algorithms. It needs a model of its objects, the responsibilities of those objects, their relationships and interactions, and a structure that allows the system to be changed safely.

Core distinction: DSA provides techniques for solving computational problems. LLD defines how the application's code and objects are organised so that those techniques work together as part of a usable software product.

3. Running example: Quick Ride

The lecture invents a ride-booking company called **Quick Ride**, similar in purpose to platforms such as Ola or Uber. A user enters a source and destination, the platform assigns a rider, and the rider transports the user to the destination.

Two new employees are used to contrast DSA-only thinking with LLD-aware thinking:

Person	Knowledge	Initial approach
Anurag	Strong DSA knowledge; has not studied LLD.	Treats building the application mainly as solving algorithmic problems.

Person	Knowledge	Initial approach
Maurya	Knows DSA and also understands LLD.	Designs the application's structure first, then considers the algorithms needed inside it.

3.1 Anurag's approach: start directly with algorithms

Anurag identifies two algorithmic problems that a ride-booking platform must solve.

Problem A — Find a route from source to destination

- Represent the city as a graph.
- Treat intersections or junctions as **nodes (vertices)**.
- Treat roads connecting intersections as **edges**.
- The city can then be modelled as a graph, with the understanding that a real city graph can be much more complicated than a simplified drawing.
- Find a shortest path from the user's source to the destination. The lecture refers to Dijkstra's algorithm for this task.

This is a valid algorithmic component: the graph represents the road network, and a shortest-path algorithm helps identify a route. But it is only one part of the application.

Problem B — Assign a nearby rider to the user

- A user is at a location, and several riders may be nearby.
- Anurag considers keeping the riders associated with a user in a priority queue implemented using a heap.
- With a suitable priority key—such as distance—the closest eligible rider can be selected first.
- The lecture describes a min-heap, where the smallest priority value is at the top; a max-heap instead places the largest value at the top.

This is another algorithmic component. In an actual implementation, the system would need to define how distance is calculated, how rider availability is checked, and how the priority queue is updated as locations and statuses change.

> **Important clarification:** A priority queue does not automatically know which rider is closest. The application must supply a meaningful priority value (for example, estimated distance or pickup time) and maintain it as conditions change. This clarification makes the example precise without changing the lecture's central point.

3.2 Why Anurag's solution is incomplete

When Anurag presents his algorithms, the manager explains that he has solved some problems but has not yet described the application itself. The manager asks questions such as:

- What objects or entities exist in the application?
- What relationships exist between those objects?
- How will the objects interact with one another?
- How will sensitive data be protected?
- How will notifications be integrated—for example, to tell a user about ride status or distance?
- How will a payment gateway be integrated?
- How will the application continue to work when it serves millions of users?

Anurag jumped directly to the DSA portion. His route-finding and rider-selection algorithms may be useful, but they do not answer the wider design questions above.

3.3 Maurya's approach: design the structure first

Maurya first asks what the application consists of and how its parts fit together. The lecture identifies these possible objects or entities:

- **User:** the person who books a ride.
- **Rider:** the person assigned to fulfil the ride.
- **Location:** information about where a user or rider is and where a trip begins or ends.
- **Notification:** a component for communicating updates.
- **Payment:** a component for handling payment-related behaviour or integration.

Maurya then considers how the objects are related—for example, how a user is connected to a ride and rider, and how location information is used. The exact class design and relationships are topics to be worked out later; the lecture is introducing the need to identify them.

3.4 Security and growth are part of the design

- The application may contain sensitive information, such as users' and riders' mobile numbers.
- The lecture gives the example of hiding or masking phone numbers so that, after a ride is complete, neither party necessarily needs to see the other person's real number.
- The code should be structured with the expectation that the platform may grow to millions of users.
- Only after the application's structure and major design concerns have been considered does Maurya turn to the algorithmic tasks Anurag identified.

> **Lesson from Quick Ride:** LLD does not replace DSA. It provides the structure in which the right algorithms, services, and interactions can be used.

4. The three pillars of LLD

The lecture emphasises three qualities that a good low-level design should support.

Pillar	Meaning	Question to ask
Scalability	The application should continue to handle growth in users or workload. The lecture also connects this to how easily new capabilities can be integrated as the system grows.	Can the design handle more users and more work without breaking down?
Maintainability	The code should be understandable, testable and debuggable, and new features should be added with limited risk to existing features.	Can we change or debug one part without causing problems elsewhere?
Reusability	Components and algorithms should be reusable in other applications with little modification, rather than being tightly bound to one product.	Can we plug this component into another suitable application?

4.1 Scalability

Scalability concerns how well an application sustains increased demand. In the Quick Ride story, the platform may begin with relatively few users and later need to support millions.

- The design should not assume that the application will always have a small number of users.
- As demand grows, the system must continue to provide its intended behaviour instead of failing under the load.
- The teacher also links scalability with the effort needed to integrate new features and grow the application.

A useful study distinction: the lecture uses scalability in a broad way. In system-design discussions, the ability to handle more traffic is often distinguished from code extensibility, although both matter when designing a system that can grow.

4.2 Maintainability

Maintainability describes how easy the code is to modify, extend, understand, and debug over time.

- **Safe feature changes:** adding a new feature should not cause several existing features to fail.
- **Easy integration:** the code should be organised so that adding new functionality requires limited effort.
- **Debuggability:** developers should be able to investigate the code, check its behaviour, and locate defects without excessive difficulty.
- **Fewer defects:** the teacher notes that a real application cannot realistically be guaranteed to have zero bugs, so the goal is to minimise bugs and make them easier to detect and fix.

Example: if adding a payment option breaks ride booking, notifications, and user profiles, the code is difficult to maintain. Good separation of responsibilities and clear interfaces reduce this kind of risk.

4.3 Reusability

Reusability means that a component can be used again in another suitable context rather than being written from scratch or being inseparably tied to one application.

- Notifications and payment integrations are not unique to ride-booking applications; many products need similar capabilities.
- A notification component designed with a clear interface may be reused in a food-delivery application or another service.
- The same idea applies to rider/delivery-partner assignment. Similar assignment problems appear in ride-hailing, food delivery, e-commerce delivery, and quick-commerce services.
- The details may differ, but the general task—assigning a suitable person or partner to a user, order, or restaurant—can share a reusable algorithm or component.

The lecture calls this a **plug-and-play model**: a component should be easy to connect to another application and should work without extensive rewriting.

4.4 Tight coupling

The lecture warns that code should not be tightly coupled to one specific application. Tight coupling means that one component depends heavily on the internal details of another component, making it difficult to change or reuse independently.

- If a notification module can only work inside one particular ride-booking codebase, it is less reusable.
- If the rider-matching algorithm is written so that it assumes every detail of Quick Ride, moving it to a food-delivery platform may require substantial changes.
- Clear interfaces and separation of responsibilities make it easier to reuse components.

> **Practical rule:** Keep reusable logic focused on its responsibility. Supply application-specific details through parameters, interfaces, or configuration rather than hard-coding them throughout the component.

5. Reusable algorithms: rider matching example

The teacher returns to the rider-mapping algorithm to show why reusability is broader than copying and pasting code.

In Quick Ride, the system chooses a rider for a user. In a food-delivery platform, the system may assign a delivery partner to an order or restaurant. E-commerce and quick-commerce platforms also need to assign delivery personnel to orders.

Although the objects and business rules differ, the underlying assignment problem can be similar: select an appropriate available person or partner based on relevant criteria.

Design implication

- Keep the general matching or ranking logic separate from ride-booking-specific details.
- Allow the caller to provide the candidates, eligibility rules, distance/time values, and other relevant inputs.
- Return an assignment or ranked choice without assuming that the caller is necessarily Quick Ride.
- Keep platform-specific actions—such as sending a ride confirmation or updating a food order—in the relevant application layer.

This is an illustrative design interpretation of the lecture's example, not code shown in the transcript.

> **Remember:** Reusability is not simply duplicating the same code in multiple places. It is designing a component so it can serve multiple appropriate use cases with minimal changes.

6. LLD versus HLD

HLD means High-Level Design. **LLD** means Low-Level Design. The lecture says students often confuse the two, so it contrasts the decisions made at each level.

Aspect	HLD — High-Level Design	LLD — Low-Level Design
Main focus	Overall system architecture.	Internal code structure and object-level design.
Typical questions	What services, servers, databases, and major components are needed? How will they communicate and scale?	What classes/objects exist? What responsibilities and methods do they have? How do they relate and collaborate?
Technology stack	May choose a stack such as Java with Spring Boot.	Focuses on how the code and objects are organised within the chosen system.
Database	Considers the database technology or broad storage architecture, such as SQL, NoSQL, or a hybrid approach.	May define application-level entities and how the code interacts with persistence, depending on scope.
Scaling and cost	Considers server scaling and infrastructure cost optimisation.	Supports maintainable, reusable code; it is not a substitute for infrastructure-level capacity planning.
Typical artefacts	System architecture diagrams and major component/service diagrams.	Class diagrams, object relationships, interfaces, and method-level design.
Coding during an interview	Usually little or no implementation code; the emphasis is on architecture and trade-offs.	More detailed object/class design and often code-level reasoning.

6.1 HLD topics named in the lecture

- **Technology stack:** the technologies used to build the application, such as Java and Spring Boot.
- **Database choice:** whether SQL, NoSQL, or a combination is appropriate. The teacher notes that SQL and NoSQL are examples, not the only database options.
- **Server scaling:** how deployment capacity can expand when traffic increases.
- **Cost optimisation:** infrastructure costs should respond to demand. If traffic rises, capacity may need to increase; if traffic falls, unnecessary servers or resources should be reduced where possible.
- **System architecture:** deciding which major components exist and how requests and data move through them.

6.2 LLD topics named or implied in the lecture

- Objects/entities and their responsibilities.
- Relationships and interactions between objects.
- Class diagrams and code structure.

- Integration of behaviour such as notifications and payments.
- Security-conscious handling of sensitive information.
- Designing code for maintainability and reuse.

The levels complement one another. A well-designed class structure cannot by itself solve every infrastructure-scaling problem, and a good server architecture does not automatically make the code easy to maintain.

7. DSA, LLD, HLD, and the application: how they fit together

Concept	Main question	Contribution
DSA	How can this computational problem be solved efficiently?	Provides data structures and algorithms, such as graphs, heaps, and shortest-path algorithms.
LLD	How should the application's code-level parts be structured?	Defines objects, relationships, responsibilities, interfaces, and interactions.
HLD	How should the overall system be organised and deployed?	Defines major components, technology choices, data stores, servers, scaling strategy, and broad architecture.
Application	How do all these pieces work together to deliver a useful product?	Combines architecture, code structure, algorithms, integrations, data, and operational concerns.

The lecture's memorable analogy: "DSA is the brain of an application; LLD is its skeleton." In other words, algorithms provide problem-solving capability, while LLD provides the structure that organises the application. HLD supplies the broader system architecture around that structure.

8. Common misunderstandings to avoid

- **Misunderstanding 1:** "If I know DSA, I can design the whole application." DSA is essential, but it does not specify the complete object model, interactions, integrations, security, or architecture.
- **Misunderstanding 2:** "LLD is the same as HLD." HLD concerns the overall system; LLD concerns code-level structure and collaboration.
- **Misunderstanding 3:** "A priority queue automatically finds the closest rider." The application must define and update the priority value, and account for eligibility and availability.
- **Misunderstanding 4:** "Reusable code is just copied code." Reusable code is designed to work in multiple contexts through clear boundaries and minimal application-specific assumptions.
- **Misunderstanding 5:** "Maintainability means there will be no bugs." The goal is to make code easier to change and debug and to reduce defects, not to promise a bug-free system.
- **Misunderstanding 6:** "LLD alone handles all scaling." LLD contributes to code quality and extensibility, while HLD also addresses infrastructure, server capacity, and cost.

9. Quick revision sheet

Term	One-line revision
LLD	Design of the internal code structure, objects, relationships, and interactions of an application.
DSA	Data structures and algorithms used to solve individual computational problems.
HLD	High-level system architecture, including major components, technologies, databases, deployment, scaling, and cost.
Node / vertex	A point in a graph; in the city example, an intersection.
Edge	A connection between graph nodes; in the city example, a road.
Shortest-path algorithm	An algorithm used to find a route between graph nodes; Dijkstra's algorithm is named in the lecture.
Priority queue / heap	A data structure that returns an item according to its priority; a min-heap exposes the smallest key first.
Scalability	Ability to sustain increased demand or workload.
Maintainability	Ease of understanding, changing, testing, and debugging the code.
Reusability	Ability to use a component or algorithm again in another suitable context.
Tight coupling	Strong dependence between components that makes changes and reuse harder.
Plug-and-play	A component can be connected to another suitable system with little modification.

10. Practice questions

- **1.** Define LLD in your own words. How is it different from DSA?
- **2.** Why were Anurag's two algorithms insufficient to describe the Quick Ride application?
- **3.** How can a city road network be represented as a graph? What do nodes and edges represent?
- **4.** How could a priority queue help select a nearby rider? What information must be used as the priority?
- **5.** Name five objects or entities that Maurya identifies or considers for Quick Ride.
- **6.** Why should a ride-booking platform think about privacy and phone-number masking?
- **7.** Explain scalability, maintainability, and reusability with one example each.
- **8.** What does tight coupling mean, and why can it make reuse difficult?
- **9.** How can a rider-matching or delivery-partner assignment algorithm be reused across ride-hailing and food delivery?
- **10.** Compare HLD and LLD in terms of focus, decisions, and typical diagrams.
- **11.** What is server scaling? How does cost optimisation relate to traffic changes?
- **12.** Explain the analogy: "DSA is the brain; LLD is the skeleton." Where does HLD fit?

11. Suggested answers for self-check

LLD vs DSA: DSA solves individual computational problems. LLD organises objects, relationships, and code responsibilities so multiple behaviours work together in a complete application.

Anurag's gap: He discussed shortest-path and rider-selection algorithms but not the application's entities, relationships, integrations, security, or ability to support many users.

City graph: Intersections are nodes/vertices; roads between intersections are edges. A shortest-path algorithm can then find a route through the graph.

Priority queue: Store eligible rider candidates with a meaningful priority such as estimated distance or pickup time. The queue can expose the best-ranked candidate, provided priorities and availability are maintained.

Quick Ride entities: User, rider, location, notification, and payment are named in the lecture.

Privacy: Users and riders should not necessarily need access to each other's real phone numbers after a ride. Masking helps protect sensitive personal information.

Three pillars: Scalability: withstand more users. Maintainability: add a feature without breaking existing features. Reusability: reuse a notification or assignment component in another suitable application.

Tight coupling: A component depends heavily on one application's internal details, so moving or changing it requires many modifications.

Assignment reuse: Separate general candidate-ranking/assignment logic from product-specific rules and actions; provide relevant inputs through clear interfaces.

HLD vs LLD: HLD defines the overall system architecture and infrastructure. LLD defines internal objects, classes, responsibilities, and interactions.

Scaling and cost: Server scaling adjusts capacity to demand. Cost optimisation avoids paying for unnecessary resources when demand is lower while retaining enough capacity for actual load.

Brain/skeleton analogy: DSA supplies problem-solving logic; LLD supplies the structure that organises the code. HLD describes the broader architecture in which the application runs.

12. Lecture wrap-up and next topic

The teacher closes by reiterating that HLD, LLD, and DSA work together to build an application. HLD focuses on system architecture; LLD focuses on the code's structure; DSA supplies techniques for solving specific problems. The lecture previews the next lesson on OOP concepts and the pillars of OOP.

> **Final takeaway:** Do not jump straight from a product requirement to an algorithm. First understand the system and its objects, relationships, interactions, security needs, and change requirements; then choose and integrate the algorithms that solve the relevant subproblems.

Source note: These notes are based on the lecture transcript supplied in this conversation. The transcript contains automatic transcription errors in some names and technical words; obvious terms have been standardised for readability (for example, "Dijkstra's algorithm"). Explanatory clarifications are identified where useful and are not presented as verbatim teacher statements.